
UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e Tecnologia

Departamento de Computação

Laboratório de Arquiteturas de Alto Desempenho

Prática 6 — Jogo da Vida de Conway em FPGA
com Arquitetura Pipeline e Saída VGA

Professor: *Dr. Emerson Carlos Pedrino*

Pedro Augusto Faria (RA: 821124)

Curso: *Engenharia da Computação*

1 Introdução

O Jogo da Vida (*Game of Life*), criado pelo matemático britânico John Horton Conway em 1970, é um autômato celular bidimensional que simula a evolução de uma população de células dispostas em uma grade regular. Apesar de sua formulação extremamente simples — quatro regras de transição aplicadas a uma vizinhança de 8 células — o sistema exibe comportamento emergente de notável complexidade, incluindo estruturas estáveis, osciladores, planadores (*gliders*) e até mesmo estruturas computacionalmente universais (capazes de simular uma máquina de Turing).

Do ponto de vista das arquiteturas de alto desempenho, o Jogo da Vida é um problema paradigmático para o estudo de paralelismo espacial e temporal. A atualização de cada célula depende exclusivamente dos valores de seus 8 vizinhos no ciclo anterior, sem qualquer dependência de dados entre células distintas na mesma geração. Esta independência torna o problema *embaraçosamente paralelo* e ideal para implementação em hardware reconfigurável (FPGA), onde todas as células podem ser avaliadas simultaneamente em um único ciclo de clock, em contraste com uma implementação sequencial em software que percorreria as $64 \times 64 = 4096$ células uma a uma.

Esta prática implementa o Jogo da Vida em uma grade de 64×64 células na FPGA Altera DE1 (Cyclone II), utilizando o ambiente Quartus Prime. O sistema exibe a evolução do autômato em tempo real em um monitor VGA de resolução 640×480 pixels. O usuário pode controlar a velocidade de evolução (automática ou manual por botão), selecionar o padrão inicial (padrão *hardcoded* ou carregado de ROM) e avançar geração a geração interativamente.

1.1 Fundamentação Teórica

1.1.1 Autômatos Celulares e o Jogo da Vida

Um autômato celular é um modelo computacional discreto no espaço, no tempo e nos estados. O Jogo da Vida de Conway opera sobre uma grade bidimensional de células, cada uma podendo estar *viva* (1) ou *morta* (0). A cada geração (passo de tempo), todas as células são atualizadas simultaneamente de acordo com as seguintes regras, baseadas no número n de vizinhos vivos na vizinhança de Moore (os 8 pixels adjacentes):

1. Uma célula **viva** com $n < 2$ vizinhos vivos **morre** (subpopulação);
2. Uma célula **viva** com $n \in \{2, 3\}$ vizinhos vivos **sobrevive** para a próxima geração;
3. Uma célula **viva** com $n > 3$ vizinhos vivos **morre** (superpopulação);
4. Uma célula **morta** com $n = 3$ vizinhos vivos **nasce** (reprodução).

Estas quatro regras podem ser expressas compactamente como:

$$c'(i, j) = \begin{cases} 1 & \text{se } c(i, j) = 1 \text{ e } n(i, j) \in \{2, 3\} \\ 1 & \text{se } c(i, j) = 0 \text{ e } n(i, j) = 3 \\ 0 & \text{caso contrário} \end{cases} \quad (1)$$

onde $c(i, j)$ é o estado da célula na linha i , coluna j na geração atual, $c'(i, j)$ é seu estado na próxima geração, e $n(i, j)$ é o número de vizinhos vivos:

$$n(i, j) = \sum_{\substack{l \in \{-1, 0, 1\} \\ k \in \{-1, 0, 1\} \\ (l, k) \neq (0, 0)}} c(i+l, j+k) \quad (2)$$

1.1.2 Padrão Inicial: Canhão de Planadores de Gosper

O padrão *hardcoded* implementado no sistema é o **Canhão de Planadores de Gosper** (*Gosper Glider Gun*), descoberto por Bill Gosper em 1970 em resposta a um desafio proposto pelo próprio Conway. É o primeiro padrão descoberto com crescimento ilimitado de população: a estrutura oscila periodicamente com período 30 e emite um novo planador (*glider*) a cada 30 gerações, resultando em crescimento linear da população ao longo do tempo.

O planador (*glider*) é a menor estrutura periódica que se translada pelo espaço: completa um ciclo em 4 gerações e desloca-se 1 célula na diagonal a cada ciclo, com velocidade de $c/4$ (um quarto da velocidade máxima de propagação de informação na grade). O canhão de Gosper ocupa uma região de aproximadamente 36×9 células e é composto por duas estruturas oscilantes (*pentadecathlons* modificados) e dois blocos estáticos que refletem os planadores gerados.

1.1.3 Representação da Grade em Hardware

A grade 64×64 é armazenada como um vetor de 4096 bits: `current_grid[4095:0]`, onde a célula na linha i (0-indexada) e coluna j está no bit de índice $i \times 64 + j$. Esta representação planificada (*row-major*) é natural para registradores em FPGA e permite que o acesso a qualquer célula seja realizado por aritmética de índice simples, sintetizável em lógica combinacional.

1.1.4 Divisão de Clock e Controle de Velocidade

O clock principal da placa DE1 é de 50 MHz, correspondendo a um período de 20 ns. Para que o Jogo da Vida evolua em velocidade visível ao olho humano (da ordem de 0,1 a 10 gerações por segundo), é necessário dividir este clock por um fator elevado.

O módulo `clock_divider` conta 5.000.000 ciclos de clock (0,1 segundo) antes de emitir um pulso de atualização (`update_pulse`) de 1 ciclo de duração. Isso resulta em uma taxa de atualização de 10 gerações por segundo no modo automático.

No modo manual, o botão `KEY[1]` gera um pulso a cada acionamento por meio do módulo `button_pulse`, que detecta a transição de descida (*falling edge*) do botão. O módulo `pulse_mux` seleciona entre os dois modos de operação por meio do *switch* `SW[1]`.

1.1.5 Mapeamento da Grade para o Monitor VGA

A grade 64×64 é exibida no monitor VGA 640×480 com cada célula ocupando uma região retangular de 10×7 pixels (largura $\times$ altura). Isso resulta em uma área de exibição de 640×448 pixels, que cabe dentro do quadro VGA ativo. O módulo `LifeToVGAMono2` mapeia as coordenadas de pixel do controlador VGA para o índice de célula correspondente na grade:

$$\text{col_index} = \left\lfloor \frac{\text{pixel_column}}{10} \right\rfloor, \quad \text{row_index} = \left\lfloor \frac{\text{pixel_row}}{7} \right\rfloor \quad (3)$$

O pixel de saída recebe o valor da célula `grid[row_index  $\times$  64 + col_index]`. Fora da região de interesse (`pixel_column  $\geq$  640` ou `pixel_row  $\geq$  448`), o pixel de habilitação (`pixel_enable`) é desativado e a saída é forçada a 0.

1.1.6 Carregamento de Padrão por ROM

Quando `SW[0] = 0` (modo ROM), o sistema carrega o padrão inicial da memória ROM (`rom`) após o reset. O módulo `GameOfLife2` entra no estado de carregamento (`loading = 1`) e percorre sequencialmente os 4096 endereços da ROM, copiando cada bit para a posição correspondente em `current_grid`. O processo leva 4096 ciclos de clock ($\approx 82 \mu\text{s}$ a 50 MHz) e é completamente transparente ao usuário.

1.2 Objetivos

- Implementar o Jogo da Vida de Conway em uma grade 64×64 em hardware Verilog, executando na FPGA Altera DE1;
- Aplicar a arquitetura pipeline para atualização do estado da grade a cada geração, com sincronismo controlado por divisor de clock;
- Exibir a evolução do autômato em tempo real em monitor VGA 640×480 , com mapeamento correto de células para pixels;

- Implementar controle interativo: modo automático (10 gen/s) e manual (avanço por botão), seleção de padrão inicial (ROM ou *hardcoded*), e reset por botão;
- Demonstrar o comportamento do Canhão de Planadores de Gosper como padrão *hardcoded* e de padrões arbitrários carregados de ROM.

1.3 Metodologia

O projeto foi desenvolvido no ambiente Quartus Prime (Versão 24.1std.0 Build 1077, Lite Edition), com módulos descritos em Verilog e integrados por diagrama esquemático BDF. O módulo VGA_SYNC foi fornecido pelo professor. Os módulos desenvolvidos foram sintetizados para a FPGA Cyclone II EP2C20F484C7 da placa Altera DE1, com pinagem definida pelo arquivo de pinos oficial.

2 Desenvolvimento Teórico

2.1 Análise das Regras do Jogo da Vida em Hardware

A implementação das regras do Jogo da Vida em hardware digital requer, para cada célula (i, j) , o cálculo da soma dos 8 vizinhos e a avaliação da função de transição. Para uma grade binária (células de 1 bit), a soma dos 8 vizinhos é um inteiro no intervalo $[0, 8]$, representável em 4 bits. A função de transição é então:

$$c'(i, j) = [c(i, j) \wedge (n = 2 \vee n = 3)] \vee [\neg c(i, j) \wedge (n = 3)] \quad (4)$$

que simplifica para:

$$c'(i, j) = (n = 3) \vee [c(i, j) \wedge (n = 2)] \quad (5)$$

Esta expressão booleana é sintetizável diretamente em lógica combinacional: um somador de 8 bits de 1 bit cada, seguido de um comparador e duas portas lógicas. Em hardware paralelo, todas as 4096 células são avaliadas simultaneamente, completando uma geração em um único ciclo de clock (após a latência do combinacional).

Na implementação em Verilog adotada, o laço `for` aninhado sobre todas as células é expandido pelo sintetizador em lógica paralela para a grade completa, confirmando o caráter massivamente paralelo da arquitetura.

2.2 Condições de Fronteira

A grade 64×64 implementada utiliza **fronteiras fixas**: as células das bordas (linha 0, linha 63, coluna 0 e coluna 63) são mantidas no estado da geração anterior sem aplicar

as regras de transição. Esta é uma simplificação de implementação que evita o tratamento de casos especiais de vizinhança nas bordas e a lógica de *wrapping* toroidal. Para o Canhão de Gosper posicionado no interior da grade (linhas 10–20, colunas 0–35), a borda fixa não afeta o comportamento do padrão por muitas gerações, até que os planadores gerados atinjam a borda e sejam absorvidos.

2.3 Arquitetura de Atualização: Paralelismo Massivo vs. Pipeline

Existem dois paradigmas principais para implementar a atualização do Jogo da Vida em FPGA:

Paralelo puro: todas as células são calculadas simultaneamente em lógica combinacional, e o resultado é registrado em um único flip-flop de habilitação (`update_pulse`). Esta é a abordagem adotada neste projeto. O custo é elevado em termos de lógica combinacional (4096 somadores + 4096 comparadores), mas o throughput é máximo: 1 geração por ciclo de clock habilitado.

Pipeline seriado: as células são calculadas em sequência por um único núcleo de processamento com *line buffers*, similar ao pipeline da Prática anterior. O custo de lógica é mínimo, mas o throughput é de 1 célula por ciclo, exigindo 4096 ciclos por geração.

O projeto utiliza a abordagem paralela pura, onde o vetor `next [4095:0]` é calculado inteiramente em lógica combinacional dentro do bloco `always` e registrado atomicamente no flanco de subida do clock quando `update_pulse` é ativo. A taxa de atualização é controlada externamente pelo `clock_divider`, que determina a cadência das gerações.

2.4 Sincronismo entre Atualização da Grade e Varredura VGA

Um aspecto crítico do projeto é a coexistência de dois processos assíncronos: a atualização da grade (a 10 Hz ou sob controle manual) e a varredura VGA (a 60 Hz, pixel clock ≈ 25 MHz). Ambos leem `current_grid` simultaneamente, mas apenas o módulo `GameOfLife2` escreve neste registro.

A atualização ocorre em um único ciclo de clock (escrita síncrona no flanco de subida quando `update_pulse = 1`), enquanto a leitura pelo módulo `LifeToVGAMono2` é combinacional (sem clock). Como a taxa de atualização (10 Hz) é vastamente inferior à taxa de quadros VGA (60 Hz), a probabilidade de uma atualização ocorrer durante a varredura de um quadro é de aproximadamente $1/(50\text{MHz}/10\text{Hz}) \approx 2 \times 10^{-7}$ por ciclo, tornando artefatos visuais de *tearing* praticamente imperceptíveis.

2.5 Mapeamento de Escala para VGA

O fator de escala inteiro (10 pixels por coluna, 7 pixels por linha) foi escolhido para maximizar o uso da tela VGA 640×480 :

$$64 \times 10 = 640 \text{ pixels de largura} \quad (\text{uso total}) \quad (6)$$

$$64 \times 7 = 448 \text{ pixels de altura} \quad (32 \text{ pixels não utilizados}) \quad (7)$$

A divisão inteira por potências de 2 não é aplicável diretamente (10 e 7 não são potências de 2), de forma que o Quartus Prime sintetiza divisores combinacionais para estas constantes. Como as entradas `pixel_column` e `pixel_row` são sinais de 10 bits com valores máximos de 639 e 479, os quocientes `col_index` e `row_index` têm no máximo 6 bits (0 a 63), cobrindo exatamente o intervalo de índices da grade.

3 Desenvolvimento Prático

3.1 Ambiente de Desenvolvimento

Tabela 1: Especificações do ambiente de desenvolvimento.

Parâmetro	Valor
Placa FPGA	Altera DE1 (Cyclone II EP2C20F484C7)
Ferramenta EDA	Quartus Prime 24.1std.0 Build 1077 (Lite Edition)
Linguagem principal	Verilog
Linguagem auxiliar	VHDL (VGA_SYNC, fornecido)
Clock da placa	50 MHz
Pixel clock VGA	$\approx 25,175$ MHz
Resolução VGA	640×480 @ 60 Hz
Resolução da grade	64×64 células, 1 bit/célula
Tamanho da célula na tela	10×7 pixels (col × lin)
Taxa de atualização	10 gerações/s (automático) ou manual
Sistema Operacional	Ubuntu 24.04.4 LTS (Kubuntu)

3.2 Estrutura Geral do Projeto e Diagrama de Blocos

O sistema é integrado pelo módulo *top-level vga_filtro* por meio do editor de esquemáticos BDF do Quartus Prime. A Figura 1 apresenta o diagrama BDF representativo do sistema completo.

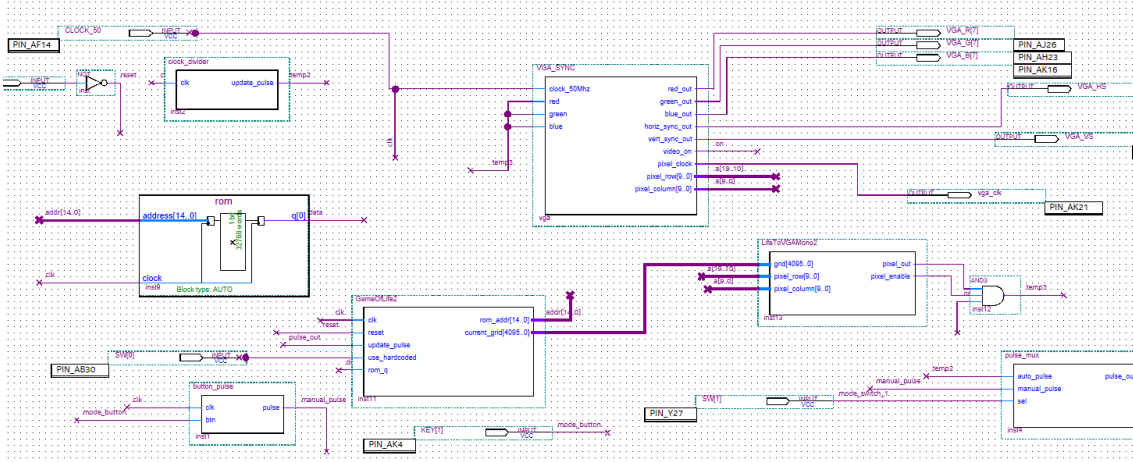


Figura 1: Diagrama BDF do módulo *top-level* `vga_filtro` no Quartus Prime, evidenciando a interligação de todos os componentes do sistema do Jogo da Vida.

Os módulos que compõem o sistema são:

- `VGA_SYNC` — Controlador VGA (VHDL, fornecido); gera HSync, VSync, `video_on`, `pixel_clock` e os contadores de coordenadas `pixel_column/pixel_row`;
- `GameOfLife2` — Núcleo do autômato celular; armazena e atualiza a grade 64×64 , com suporte a reset, carregamento de ROM e padrão *hardcoded*;
- `LifeToVGAMono2` — Conversor grade $\rightarrow$ VGA; mapeia as coordenadas de pixel para o índice de célula e habilita a saída;
- `clock_divider` — Divisor de clock; gera pulso de atualização a 10 Hz (a cada 5.000.000 ciclos de 50 MHz);
- `button_pulse` — Detector de borda de descida do botão; gera pulso de 1 ciclo por acionamento de `KEY[1]`;
- `pulse_mux` — Multiplexador de pulso; seleciona entre atualização automática e manual por `SW[1]`;
- `rom` — Memória ROM com padrão inicial alternativo de 4096 bits (1 bit por endereço).

3.3 Módulo `GameOfLife2` — Núcleo do Autômato Celular

O módulo `GameOfLife2` é o componente central do sistema. Ele armazena o estado completo da grade 64×64 no registrador `current_grid[4095:0]` e implementa três modos de operação: inicialização com padrão *hardcoded* (Canhão de Gosper), carregamento da ROM e atualização pela regra do Jogo da Vida.


```

1      module GameOfLife2 (
2          input clk,
3          input reset,
4          input update_pulse,
5          input use_hardcoded,
6          input rom_q,
7          output reg [14:0] rom_addr,
8          output reg [4095:0] current_grid
9      );
10     integer i, j, l, c, count;
11     reg [4095:0] next;
12     reg [11:0] load_addr;
13     reg loading;
14
15     task set_hardcoded;
16     begin
17         current_grid = 0;
18         // Canhao de Planadores de Gosper
19         current_grid[10*64 + 24] = 1;
20         current_grid[11*64 + 22] = 1; current_grid[11*64 + 24] = 1;
21         current_grid[12*64 + 12] = 1; current_grid[12*64 + 13] = 1;
22         current_grid[12*64 + 20] = 1; current_grid[12*64 + 21] = 1;
23         current_grid[12*64 + 34] = 1; current_grid[12*64 + 35] = 1;
24         current_grid[13*64 + 11] = 1; current_grid[13*64 + 15] = 1;
25         current_grid[13*64 + 20] = 1; current_grid[13*64 + 21] = 1;
26         current_grid[13*64 + 34] = 1; current_grid[13*64 + 35] = 1;
27         current_grid[14*64 + 0] = 1; current_grid[14*64 + 1] = 1;
28         current_grid[14*64 + 10] = 1; current_grid[14*64 + 16] = 1;
29         current_grid[14*64 + 20] = 1; current_grid[14*64 + 21] = 1;
30         current_grid[15*64 + 0] = 1; current_grid[15*64 + 1] = 1;
31         current_grid[15*64 + 10] = 1; current_grid[15*64 + 14] = 1;
32         current_grid[15*64 + 16] = 1; current_grid[15*64 + 17] = 1;
33         current_grid[15*64 + 22] = 1; current_grid[15*64 + 24] = 1;
34         current_grid[16*64 + 10] = 1; current_grid[16*64 + 16] = 1;
35         current_grid[17*64 + 11] = 1; current_grid[17*64 + 15] = 1;
36         current_grid[18*64 + 12] = 1; current_grid[18*64 + 13] = 1;
37         current_grid[19*64 + 22] = 1; current_grid[19*64 + 24] = 1;
38         current_grid[20*64 + 23] = 1;
39     end
40     endtask
41

```

```

42     initial begin
43         set_hardcoded;
44         loading = 0;
45         load_addr = 0;
46         rom_addr = 0;
47     end
48
49     always @(posedge clk) begin
50         if (reset) begin
51             // Resetar para padrao hardcoded ou iniciar carga da ROM
52             loading <= ~use_hardcoded;
53             load_addr <= 0;
54             rom_addr <= 0;
55             if (use_hardcoded) begin
56                 // Inicializacao sequencial por atribuicoes nao-bloqueantes
57                 current_grid <= 0;
58                 current_grid[10*64 + 24] <= 1;
59                 // ... (demais celulas do Canhao de Gosper)
60                 current_grid[20*64 + 23] <= 1;
61             end else
62                 current_grid <= 0;
63             end else if (loading) begin
64                 // Carregamento serial da ROM: 1 bit por ciclo de clock
65                 if (load_addr > 0)
66                     current_grid[load_addr - 1] <= rom_q;
67                 rom_addr <= {3'b000, load_addr};
68                 if (load_addr == 12'd4095) begin
69                     current_grid[4094] <= rom_q;
70                     loading <= 0;
71                 end else
72                     load_addr <= load_addr + 1;
73             end else if (update_pulse) begin
74                 // Atualizacao paralela: calcular proxima geracao
75                 next = 0;
76                 // Bordas fixas: copiar sem aplicar regras
77                 for (i = 0; i < 64; i = i + 1) begin
78                     next[i*64 + 0] = current_grid[i*64 + 0];
79                     next[i*64 + 63] = current_grid[i*64 + 63];
80                 end
81                 for (j = 0; j < 64; j = j + 1) begin
82                     next[0*64 + j] = current_grid[0*64 + j];

```

```

83     next[63*64 + j] = current_grid[63*64 + j];
84     end
85     // Interior: aplicar regras do Jogo da Vida
86     for (i = 1; i < 63; i = i + 1) begin
87     for (j = 1; j < 63; j = j + 1) begin
88     count = 0;
89     for (l = 0; l < 3; l = l + 1)
90     for (c = 0; c < 3; c = c + 1)
91     if (!(l == 1 && c == 1))
92     count = count +
93     current_grid[(i+l-1)*64 + (j+c-1)];
94     if (current_grid[i*64 + j])
95     next[i*64+j] = (count==2||count==3) ? 1'b1 : 1'b0;
96     else
97     next[i*64+j] = (count==3) ? 1'b1 : 1'b0;
98     end
99     end
100    current_grid <= next;
101    end
102    end
103    endmodule
104

```

Listing 1: GameOfLife2.v — Núcleo do autômato celular com suporte a reset, carregamento de ROM e atualização paralela da grade 64×64 .

O bloco `always @(posedge clk)` implementa uma máquina de estados implícita com três estados: `reset`, `loading` e `running`. No estado `running`, o bloco de atualização é ativado apenas quando `update_pulse` é alto (1 ciclo a cada 5.000.000 ou por acionamento manual), calculando a variável bloqueante `next` em lógica combinacional e registrando-a em `current_grid` por atribuição não-bloqueante.

3.4 Módulo LifeToVGAMono2 — Mapeamento Grade para VGA

O módulo `LifeToVGAMono2` realiza a conversão das coordenadas de pixel do controlador VGA para o índice de célula da grade, de forma puramente combinacional.

```

1     module LifeToVGAMono2 (
2     input [4095:0] grid,
3     input [9:0] pixel_row,
4     input [9:0] pixel_column,
5     output reg pixel_out,
6     output reg pixel_enable

```

```

7         );
8         // Cada célula ocupa 10 pixels de largura e 7 de altura
9         wire [5:0] col_index = pixel_column / 10;
10        wire [5:0] row_index = pixel_row / 7;
11
12        always @(*) begin
13            if (pixel_column < 640 && pixel_row < 448) begin
14                pixel_enable = 1'b1;
15                pixel_out = grid[row_index*64 + col_index];
16            end else begin
17                pixel_enable = 1'b0;
18                pixel_out = 1'b0;
19            end
20        end
21    endmodule
22

```

Listing 2: LifeToVGAMono2.v — Mapeamento combinacional da grade 64×64 para o quadro VGA 640×480.

O sinal `pixel_enable` é combinado com `video_on` do controlador VGA e com `pixel_out` no *top-level* para formar o sinal RGB final: `temp3 = pixel_out & pixel_enable & on`. Desta forma, apenas pixels dentro da região ativa da imagem E dentro do período de varredura ativa do VGA recebem o valor da célula correspondente.

3.5 Módulo `clock_divider` — Divisor de Clock

O módulo `clock_divider` implementa um contador de 24 bits que conta de 0 a 4.999.999 (5.000.000 ciclos) e emite um pulso de 1 ciclo a cada $5\text{M}/50\text{MHz} = 0,1\text{s}$, resultando em uma taxa de atualização de 10 gerações por segundo.

```

1        module clock_divider (
2            input wire clk,
3            output reg update_pulse
4        );
5        localparam UPDATE_INTERVAL = 24'd5_000_000;
6        reg [23:0] counter;
7
8        always @(posedge clk) begin
9            if (counter >= UPDATE_INTERVAL - 1) begin
10                counter <= 0;
11                update_pulse <= 1'b1;
12            end else begin

```

```

13         counter <= counter + 1;
14         update_pulse <= 1'b0;
15     end
16 end
17 endmodule
18

```

Listing 3: clock_divider.v — Divisor de clock para controle de velocidade de evolução do autômato.

A constante UPDATE_INTERVAL pode ser alterada para ajustar a velocidade: valores menores aceleram o autômato (ex.: 500.000 para 100 gen/s) e valores maiores o desaceleram (ex.: 50.000.000 para 1 gen/s).

3.6 Módulo button_pulse — Detector de Pulso por Botão

O módulo button_pulse detecta a borda de descida do botão KEY[1] (os botões da DE1 são ativos em baixo) por meio de um flip-flop que armazena o valor anterior do botão.

```

1     module button_pulse (
2         input  clk,
3         input  btn,
4         output pulse
5     );
6     reg btn_prev;
7     always @(posedge clk)
8         btn_prev <= btn;
9     // Borda de descida: btn_prev=1, btn=0 -> pulse=1
10    assign pulse = btn_prev & ~btn;
11 endmodule
12

```

Listing 4: button_pulse.v — Detector de borda de descida para geração de pulso manual de atualização.

Como os botões da DE1 são ativos em baixo (pressionado = 0, solto = 1), a expressão `btn_prev & ~btn` detecta a transição de $1 \rightarrow 0$, correspondente ao momento em que o botão é pressionado. O pulso tem duração de exatamente 1 ciclo de clock, independentemente do tempo de pressionamento do botão.

3.7 Módulo pulse_mux — Seleção de Modo de Atualização

O módulo pulse_mux é um multiplexador 2:1 que seleciona entre o pulso automático do clock_divider e o pulso manual do button_pulse:

```

1      module pulse_mux (
2          input  auto_pulse,
3          input  manual_pulse,
4          input  sel,
5          output pulse_out
6      );
7      assign pulse_out = sel ? manual_pulse : auto_pulse;
8      endmodule
9

```

Listing 5: pulse_mux.v — Seleção entre modo de atualização automático e manual.

Com SW[1] = 0, o sistema opera em modo automático (10 gen/s). Com SW[1] = 1, cada acionamento de KEY[1] avança uma geração, permitindo observar a evolução célula a célula.

3.8 Módulo *Top-Level* vga_filtro

O módulo *top-level* vga_filtro interliga todos os componentes, mapeia os pinos físicos da DE1 e define os sinais de controle conforme a Tabela 2.

Tabela 2: Mapeamento de entradas de controle da placa DE1.

Pino	Sinal	Função
KEY[0]	reset	Reset do sistema (ativo em baixo)
KEY[1]	mode_button	Avança 1 geração (modo manual)
SW[0]	use_hardcoded	1 = Gosper Gun; 0 = carrega ROM
SW[1]	mode_switch_1	0 = automático; 1 = manual

```

1      module vga_filtro(
2          CLOCK_50,
3          KEY,
4          SW,
5          VGA_HS,
6          VGA_VS,
7          vga_clk,
8          VGA_B,
9          VGA_G,

```

```

10         VGA_R
11     );
12     input wire      CLOCK_50;
13     input wire [1:0] KEY;
14     input wire [1:0] SW;
15     output wire     VGA_HS;
16     output wire     VGA_VS;
17     output wire     vga_clk;
18     output wire [7:7] VGA_B;
19     output wire [7:7] VGA_G;
20     output wire [7:7] VGA_R;
21
22     wire [19:0]  a;                // {pixel_row,
pixel_column}
23     wire [14:0]  addr;            // endereco da ROM (15
bits)
24     wire         clk;             // clock 50 MHz
25     wire         data;            // dado lido da ROM
26     wire         manual_pulse;    // pulso do botao
27     wire         mode_button;     // entrada do botao
KEY[1]
28     wire         mode_switch_1;   // SW[1]: modo
auto/manual
29     wire         on;              // video_on do VGA
30     wire         pclk;            // pixel clock VGA
31     wire         pulse_out;       // pulso de
atualizacao ativo
32     wire         reset;           // reset: ~KEY[0]
33     wire         temp2;           // pulso automatico
(10 Hz)
34     wire         temp3;           // pixel RGB final
35     wire         SYNTHESIZED_WIRE_0; // pixel_out do
conversor VGA
36     wire         SYNTHESIZED_WIRE_1; // pixel_enable do
conversor
37     wire [4095:0] SYNTHESIZED_WIRE_2; // current_grid
38
39     // Reset ativo em baixo: KEY[0] pressionado -> reset = 1
40     assign reset = ~KEY[0];
41
42     // Detector de borda do botao de avanco manual

```

```

43     button_pulse b2v_inst1(
44         .clk(clk),
45         .btn(mode_button),
46         .pulse(manual_pulse));
47
48     // Nucleo do Jogo da Vida
49     GameOfLife2 b2v_inst11(
50         .clk(clk),
51         .reset(reset),
52         .update_pulse(pulse_out),
53         .use_hardcoded(SW[0]),
54         .rom_q(data),
55         .current_grid(SYNTHESIZED_WIRE_2),
56         .rom_addr(addr));
57
58     // Pixel RGB: ativo se celula viva E pixel habilitado E
video ativo
59     assign temp3 = SYNTHESIZED_WIRE_0 & SYNTHESIZED_WIRE_1 &
on;
60
61     // Conversor grade -> VGA
62     LifeToVGAMono2 b2v_inst13(
63         .grid(SYNTHESIZED_WIRE_2),
64         .pixel_column(a[9:0]),
65         .pixel_row(a[19:10]),
66         .pixel_out(SYNTHESIZED_WIRE_0),
67         .pixel_enable(SYNTHESIZED_WIRE_1));
68
69     // Divisor de clock: pulso automatico a 10 Hz
70     clock_divider b2v_inst2(
71         .clk(clk),
72         .update_pulse(temp2));
73
74     // Mux de modo: automatico (SW[1]=0) ou manual (SW[1]=1)
75     pulse_mux b2v_inst4(
76         .auto_pulse(temp2),
77         .manual_pulse(manual_pulse),
78         .sel(mode_switch_1),
79         .pulse_out(pulse_out));
80
81     // ROM de padrao inicial alternativo

```



```
82         rom b2v_inst9(  
83             .clock(clk),  
84             .address(addr),  
85             .q(data));  
86  
87         // Controlador VGA (VHDL, fornecido pelo professor)  
88         VGA_SYNC b2v_vga(  
89             .clock_50Mhz(clk),  
90             .red(temp3),  
91             .green(temp3),  
92             .blue(temp3),  
93             .red_out(VGA_R),  
94             .green_out(VGA_G),  
95             .blue_out(VGA_B),  
96             .horiz_sync_out(VGA_HS),  
97             .vert_sync_out(VGA_VS),  
98             .video_on(on),  
99             .pixel_clock(pclk),  
100             .pixel_column(a[9:0]),  
101             .pixel_row(a[19:10]));  
102  
103         assign clk          = CLOCK_50;  
104         assign mode_switch_1 = SW[1];  
105         assign vga_clk      = pclk;  
106         assign mode_button  = KEY[1];  
107  
108         endmodule  
109
```

Listing 6: `vga_filtro.v` — Módulo *top-level* gerado pelo editor BDF do Quartus Prime, integrando todos os componentes do sistema.

3.9 Diagrama BDF do Módulo GameOfLife2 no *Top-Level*

A Figura 2 apresenta o diagrama BDF do módulo GameOfLife2 instanciado dentro do *top-level* no Quartus Prime, evidenciando suas conexões com os demais componentes do sistema.

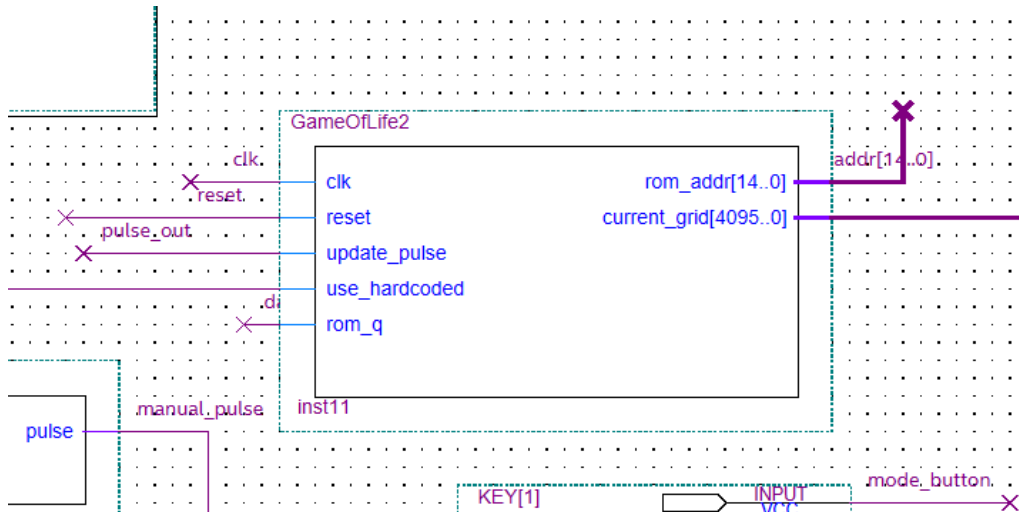


Figura 2: Diagrama BDF mostrando a instância do módulo `GameOfLife2` no *top-level*, com suas conexões de clock, reset, pulso de atualização, ROM e saída da grade.

3.10 Diagrama BDF do Módulo `VGA_SYNC` no *Top-Level*

A Figura 3 apresenta o diagrama BDF do controlador `VGA_SYNC` integrado ao *top-level*, mostrando os sinais de sincronismo e coordenadas gerados para o restante do sistema.

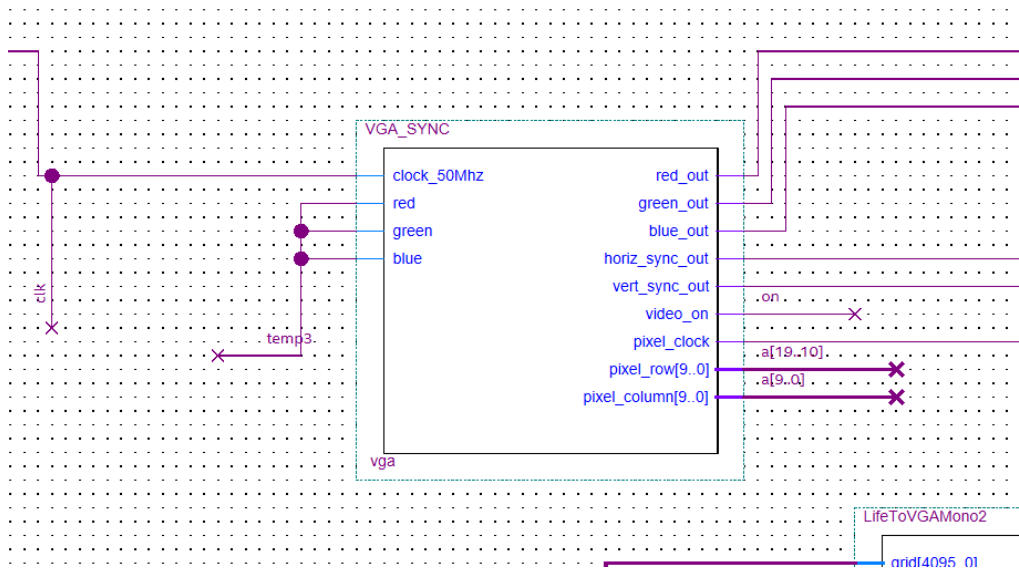


Figura 3: Diagrama BDF do controlador `VGA_SYNC` no *top-level*, evidenciando as saídas de sincronismo horizontal e vertical, pixel clock, coordenadas de varredura e sinal `video_on`.

3.11 Diagrama BDF da Memória ROM

A Figura 4 apresenta o diagrama BDF da memória ROM utilizada para armazenar o padrão inicial alternativo de 4096 células.

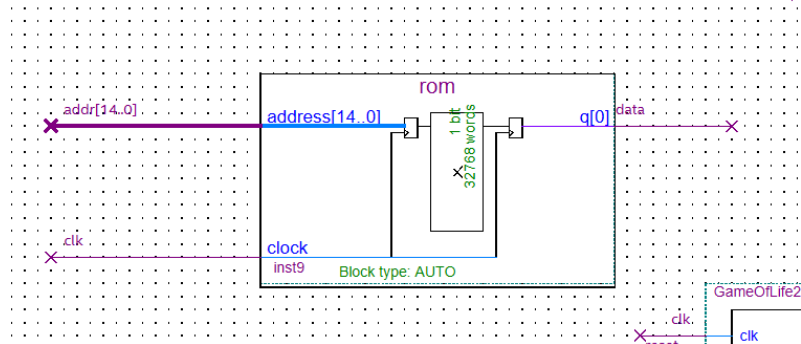


Figura 4: Diagrama BDF do módulo rom no Quartus Prime, com entrada de clock e endereço de 15 bits, e saída de dado de 1 bit por ciclo.

4 Discussão dos Resultados

4.1 Comportamento do Canhão de Planadores de Gosper

Com $SW[0] = 1$ (padrão *hardcoded*), o sistema exibe o Canhão de Planadores de Gosper imediatamente após o reset. A Figura 5 apresenta o estado do autômato capturado durante a execução na placa DE1, exibido no monitor VGA. É possível observar a estrutura central oscilante do canhão e os planadores emitidos deslocando-se diagonalmente em direção ao canto inferior direito da grade.



Figura 5: Estado do Jogo da Vida de Conway exibido no monitor VGA durante execução na placa DE1, com o Canhão de Planadores de Gosper como padrão *hardcoded* ($SW[0] = 1$). Células vivas aparecem em branco e células mortas em preto.

O comportamento observável no monitor VGA confirma as propriedades teóricas do padrão: a estrutura central oscila com período 30 gerações e emite planadores que se deslocam diagonalmente em direção ao canto inferior direito da grade. Após atingirem a borda fixa, os planadores são absorvidos e desaparecem, sem interferir com o canhão central.

O modo manual ($SW[1] = 1$) permite verificar geração a geração a evolução do padrão, confirmando que as regras de transição são aplicadas corretamente para todas as células do interior da grade.

4.2 Paralelismo Massivo e Implicações para Síntese

A abordagem de atualização paralela da grade completa em um único ciclo de clock tem implicações significativas para o processo de síntese. O sintetizador do Quartus Prime expande os laços `for` aninhados em lógica combinacional para todas as 3.844 células interiores da grade (62×62), resultando em um circuito com:

- 3.844 somadores de 8 bits de 1 bit cada (ou equivalentes com 8 portas lógicas de adição);
- 3.844 comparadores para os valores 2 e 3 do contador de vizinhos;
- 3.844 multiplexadores para a função de transição;
- 4.096 flip-flops de 1 bit para o registrador `current_grid`.

Esta lógica massiva demanda um número elevado de *Logic Elements* (LEs) da Cyclone II, possivelmente próximo ou acima da capacidade do EP2C20F484C7 (18.752 LEs). Em caso de excesso de recursos, o Quartus Prime pode não completar a síntese ou pode reduzir a frequência máxima de operação abaixo de 50 MHz. Uma solução alternativa, caso os recursos sejam insuficientes, seria adotar a abordagem de pipeline seriado com *line buffers*, similar à Prática anterior, reduzindo o custo de lógica a expensas de maior complexidade de controle.

4.3 Controle de Velocidade e Interação com o Usuário

O esquema de controle duplo (automático/manual) mostrou-se funcional e intuitivo. No modo automático, a taxa de 10 gerações por segundo é adequada para observar a dinâmica global do autômato — rápida o suficiente para perceber movimento, lenta o suficiente para identificar estruturas. O modo manual permite análise detalhada de transições específicas, essencial para verificar a correção da implementação.

O módulo `button_pulse`, ao detectar apenas a borda de descida do botão (e não o nível), elimina o problema de múltiplos acionamentos por um único pressionamento (*key bounce* em nível), embora o *bounce* em borda ainda possa causar múltiplos pulsos em um único pressionamento físico. Para uma implementação robusta, seria necessário adicionar um circuito de *debouncing* por contador.

4.4 Mapeamento VGA e Qualidade Visual

O fator de escala 10×7 pixels por célula produz células retangulares (não quadradas) na tela, o que é uma aproximação aceitável dado que o fator de aspecto ($10/7 \approx 1,43$) é próximo de 1. Células quadradas exigiria um fator de escala de 7×7 (cobrindo apenas 448×448 pixels) ou 10×10 (exigindo resolução de 640×640 , maior que o VGA

padrão). A solução adotada maximiza o uso da tela disponível dentro das restrições do padrão VGA 640×480.

4.5 Demonstração em Vídeo

O funcionamento completo do sistema — incluindo a evolução do Canhão de Planadores de Gosper em modo automático, o avanço manual geração a geração e a exibição em tempo real no monitor VGA — foi registrado em vídeo e está disponível publicamente no YouTube:

<https://youtu.be/mJRZNuSqj3Y>

O vídeo demonstra os diferentes modos de operação selecionáveis pelas chaves e botões da placa DE1, evidenciando o comportamento correto do autômato celular e o sincronismo entre a lógica de atualização e a saída VGA.

5 Conclusões

Esta prática demonstrou com sucesso a implementação do Jogo da Vida de Conway em FPGA, cobrindo desde a modelagem do autômato celular em Verilog até a exibição em tempo real no monitor VGA. As principais conclusões são:

1. **Autômatos celulares são naturalmente paralelos em FPGA:** A independência entre as atualizações de células distintas na mesma geração permite que todo o cálculo da próxima geração seja realizado em lógica combinacional paralela, completando uma geração por ciclo de clock. Esta é a exploração mais direta possível do paralelismo espacial oferecido pela arquitetura reconfigurável do FPGA.
2. **A separação de clock de atualização e clock de exibição é essencial:** A grade evolui a 10 Hz (controlada pelo `clock_divider`), enquanto o VGA varre a tela a 60 Hz com pixel clock de 25 MHz. O uso de registradores síncronos para `current_grid` e leitura combinacional pelo `LifeToVGAMono2` garante que qualquer quadro VGA exibe um estado consistente da grade, sem artefatos de escrita parcial.
3. **A interface de usuário com múltiplos modos enriquece a observação:** A combinação de reset, seleção de padrão inicial, modo automático/manual e avanço por botão transforma o sistema em uma plataforma interativa de exploração do autômato, adequada tanto para verificação de correção quanto para demonstração didática do comportamento emergente do Jogo da Vida.

4. **O Canhão de Gosper valida a implementação:** A exibição correta do canhão de Gosper — com seu período de 30 gerações e emissão regular de planadores — constitui uma validação funcional robusta da implementação das regras de Conway, pois qualquer erro nas regras de transição produziria um comportamento visivelmente incorreto deste padrão bem-documentado.
5. **A abordagem paralela pura tem custo de síntese elevado:** A expansão de laços for aninhados para 3.844 células em lógica combinacional é a principal limitação de escalabilidade do projeto. Para grades maiores (ex.: 128×128 ou 256×256), a abordagem de pipeline seriado com *line buffers* — similar à Prática anterior — seria necessária para manter o projeto dentro dos recursos disponíveis na FPGA.

6 Referências Bibliográficas

Referências

- [1] GARDNER, Martin. **Mathematical Games: The fantastic combinations of John Conway's new solitaire game "life"**. *Scientific American*, v. 223, n. 4, p. 120–123, out. 1970.
- [2] BERLEKAMP, Elwyn R.; CONWAY, John H.; GUY, Richard K. **Winning Ways for Your Mathematical Plays**. 2. ed. Natick: A K Peters, 2004. v. 4.
- [3] LIFEWIKI. **Gosper glider gun**. Disponível em: https://conwaylife.com/wiki/Gosper_glider_gun. Acesso em: 25 mai. 2026.
- [4] EMBARCADOS. **Controlador VGA — Parte 1**. Disponível em: <https://embarcados.com.br/controlador-vga-parte-1/>. Acesso em: 25 mai. 2026.
- [5] EMBARCADOS. **Controlador VGA — Parte 2**. Disponível em: <https://embarcados.com.br/controlador-vga-parte-2/>. Acesso em: 25 mai. 2026.
- [6] INTEL CORPORATION. **Quartus Prime Standard Edition Handbook**. Disponível em: <https://www.intel.com/content/www/us/en/programmable/documentation/>. Acesso em: 25 mai. 2026.
- [7] PEDRINO, Emerson Carlos. **Roteiro da Prática 6: Jogo da Vida em FPGA com Arquitetura Pipeline**. São Carlos: UFSCar — Departamento de Computação, 2026. Disponível em: <https://ava.ufscar.br>. Acesso em: 25 mai. 2026.
- [8] PATTERSON, David A.; HENNESSY, John L. **Organização e Projeto de Computadores: a Interface Hardware/Software**. 5. ed. Rio de Janeiro: Elsevier, 2017.

- [9] THOMAS, Donald E.; MOORBY, Philip R. **The Verilog Hardware Description Language**. 5. ed. New York: Springer, 2002.